

Safe Desktop-Orchestrated Server Updates

A concrete architecture for durable execution, clear status, protected cutover, and deterministic recovery in OpenTeam.

Recommendation: make the desktop the controller and viewer; make a host-resident updater the durable owner of every update job.

Audience: OpenTeam maintainers

Date: September 4, 2026

Scope: released single-host Docker Compose deployments on Linux, macOS, and Windows with Docker Desktop

Evidence base: repository inspection at commit 4c29c3a, 22 focused updater tests, and primary documentation from Electron, OpenSSH, Docker, PostgreSQL, Prisma, TUF, Sigstore, GitHub, and systemd.

Report map

- 01 Direct answer and release blockers
- 02 Current implementation and strengths
- 03 Safety gaps
- 04 Recommended architecture and control protocol
- 05 Durable state machine and commit semantics
- 06 Progress experience and version sequencing
- 07 Release metadata and alternatives
- 08 Prioritized implementation plan
- 09 Acceptance tests and sources

The decisive safety boundary is commit. Before commit, OpenTeam is quiesced and can roll back. After commit, new writes may exist and automatic rollback must stop.

Scope and assumptions

This review covers released OpenTeam installations on one host, including the documented Linux, macOS, and Windows with Docker Desktop configurations. It focuses on updates initiated from the Electron desktop app. It does not redesign the desktop app's own self-update mechanism except where server/client sequencing matters. The recommended first transport remains SSH for remote hosts; a new public privileged update API is explicitly out of scope for the first implementation.

Direct answer

Yes. OpenTeam can already update the server from the desktop app. For a matching loopback installation, Electron runs its bundled CLI. For a remote server, the app can invoke the installed `openteam update` command over strict, non-interactive SSH. If neither route is available, the UI provides a manual command. The current implementation already verifies the release Compose file with Sigstore, uses digest-pinned OpenTeam images, rejects ordinary downgrades and prereleases, serializes updates, pre-pulls images, creates a PostgreSQL dump, waits for the requested version to report ready, and rolls back configuration and the database after startup failure.

That is a good base, but it is not yet the architecture I would label "safe one-click update." The central change should be:

The desktop is a controller and viewer. A host-resident updater owns the durable job.

The desktop should submit an idempotent request, receive a job ID, and then subscribe or poll for replayable status. The host job must continue if the desktop closes, the laptop sleeps, or SSH disconnects. Every mutating step must be journaled before execution and be either idempotent or have a deterministic recovery action. The transaction should have an explicit commit point: before commit, clients and workers remain quiesced and automatic rollback is allowed; after commit, writes resume and rollback is no longer automatic.

Five items should block a claim of production-safe one-click updates:

1. Move execution ownership out of Electron and the SSH session into a durable host service or transient OS-managed job.
2. Replace production `prisma db push` with reviewed, versioned migrations and an explicit migration compatibility policy.
3. Define the full rollback unit: database, configuration, and every mutable Docker volume that a new release can change.
4. Add crash/reboot recovery through a write-ahead job journal and reconciliation state machine.
5. Keep the new stack read-only or inaccessible until health and smoke verification pass, then cross a recorded commit point before workers and writes resume.

What exists today

The update path is:

```
React Updates UI
-> constrained preload API
-> validated Electron IPC sender
-> ServerUpdater in Electron main process
    -> local: bundled CLI executed through Electron-as-Node
    -> remote: ssh -o BatchMode=yes -o StrictHostKeyChecking=yes ...
-> openteam update --json-progress
-> signed release resolution, Docker Compose pull, stop, backup, switch, start, health, rollback
```

The implementation is visible in the [desktop updater](#), [Electron IPC handlers](#), [CLI lifecycle](#), and [update safety helpers](#).

Strong foundations worth keeping

- Privilege is kept out of the renderer. The preload exposes narrow calls, inputs are validated, and IPC handlers check the sending webContents. This follows Electron's recommendation to expose narrow bridges and validate IPC senders. See the [Electron security checklist](#).
- Remote SSH uses `BatchMode=yes` and `StrictHostKeyChecking=yes`, so it neither stores a password nor silently accepts a new or changed host key. OpenSSH documents that `BatchMode` disables interactive password/passphrase prompts and strict host checking refuses unknown or changed keys. See the [OpenSSH client configuration manual](#).
- The server update request cannot pass arbitrary update flags. The version is parsed as an OpenTeam version and the SSH destination is constrained before it becomes a process argument.
- Releases use a workflow-identity Sigstore signature for the Compose document, and the release workflow replaces OpenTeam image tags with build-produced digests. Docker documents that pulling by digest pins an exact immutable image version. See [Docker pull by digest](#) and [Sigstore verification](#).
- Expensive work is ordered sensibly: download and signature verification, Compose validation, and image pulling occur before application downtime.
- An installation lock prevents two update commands from running concurrently.
- The current health check requires status `ready` and verifies the responding release version. Compose `--wait` and dependency health conditions provide a useful base; Docker documents that readiness must be represented by health checks because running is not the same as ready. See [Compose startup order](#) and [`docker compose up --wait`](#).
- Update state is written atomically to `update-state.json`, and structured progress is parsed with bounded buffers. The focused updater tests passed during this review: 22 tests, 0 failures.

Important gaps in the current path

1. Job lifetime is coupled to Electron or SSH

`ServerUpdater` owns a child process and in-memory snapshot. Remote execution is a foreground command attached to SSH; local execution is a child with stdout and stderr pipes owned by Electron. The code does not explicitly detach the update. Closing the app at minimum loses all observation and reconnection state, and can interrupt the command or its current Docker operation. Docker explicitly notes that a pull ends when the initiating client connection is lost. See [Docker pull cancellation behavior](#).

2. Persisted state is diagnostic, not a recovery protocol

The file records only the latest phase. It has no event sequence, step intent/completion pair, commit marker, rollback outcome, cancellation state, or recovery plan. After a power loss, a stale PID lock is removed, but no reconciler decides whether to resume, roll back, or verify the active version. A crash after services stop or while schema/configuration changes are in progress can leave the installation down or ambiguous.

3. Rollback covers less state than the release can mutate

The current rollback restores `compose.yaml`, `.env`, and a PostgreSQL dump. The release stack also contains mutable `computer_home`, `agent_data`, `assets`, `workspace`, `box_store`, and Caddy volumes, and init containers can move or rewrite files in those volumes. Docker treats volumes as persistent state with explicit backup and restore procedures. See [Docker volume backup and restore](#). Either those volumes must be snapshot-capable, or releases must guarantee that old code can read all mutations made before commit.

4. Production schema deployment uses a prototyping command

The migrate image currently executes `prisma db push`. Prisma describes `db push` as a prototyping workflow without migration history or control over schema/data migration, and recommends `migrate deploy` for production and testing.

See [Prisma schema prototyping](#) and [Prisma production migrations](#). This is the largest correctness risk because update rollback behavior depends on knowing exactly what schema transition occurred.

5. There is no protected commit boundary

The new Compose project starts the server and worker before final success is recorded. During verification, the new server can accept user writes and the worker can mutate state. If a later check fails and the backup is restored, those new writes disappear. A safe transaction must keep ingress and background work quiesced until verification succeeds. The moment writes resume is the commit point and must be journaled.

6. "Latest" and release immutability need stronger policy

Target artifacts are authenticated, but latest-version discovery comes from GitHub's latest-release endpoint, and the release workflow currently permits asset replacement with `--clobber` when a release already exists. GitHub now supports immutable releases that lock both the tag and assets and generate a release attestation. Enable this repository setting and use the documented draft-then-publish workflow. See [GitHub immutable releases](#).

For stronger freshness and compromise recovery, adopt TUF metadata rather than inventing a bespoke signed channel file. TUF uses signed, versioned, expiring metadata, target hashes/sizes, snapshot consistency, and role separation to address rollback, freeze, and mixed-metadata attacks. See the [TUF overview](#), [roles and metadata](#), and [security model](#).

7. The SSH host is not bound to the configured OpenTeam instance

Strict host-key checking proves that SSH reached a known host. It does not prove that `serverUrl` and `sshTarget` identify the same OpenTeam installation. A typo or reused alias can update the wrong server. The remote inspection command and authenticated server endpoint should return the same stable installation ID before the desktop enables Update.

8. Disk preflight is not capacity-aware

A fixed 4 GiB test on the installation directory does not account for database/volume snapshot size, image layer size, retention, or the separate Docker VM/storage filesystem used by Docker Desktop. Preflight should inspect both backup destination capacity and Docker capacity and calculate a conservative requirement from actual state.

Recommended architecture

Components and trust boundaries

```
Desktop renderer (untrusted UI context)
| narrow validated IPC: plan/start/status/cancel-before-cutover
Electron main process (controller)
| local CLI call or strict SSH command; never owns job lifetime
Host updater control tool
| owner-only local socket / OS service manager
Durable updater worker (outside the Compose stack)
| append-only journal + atomic snapshot + exclusive host lock
| verified release manifest + exact image digests
| Docker Compose + backup/snapshot adapters
OpenTeam workload stack
| maintenance gate -> staged start -> commit -> normal traffic
```

Do not place the first version of this control plane in the OpenTeam server container and do not expose a Docker-socket sidecar. The workload is intentionally stopped and replaced during an update, while mounting the Docker socket into a remotely reachable container creates a root-equivalent control surface. Keep the updater on the host, reachable only through owner-local IPC. Continue to use SSH as the remote transport.

On Linux, the CLI can launch a named transient systemd service or call a small installed user service. `systemd-run` services execute in a clean, detached environment with the service manager as parent, independently of the initiating shell. See `systemd-run`. Use `Type=exec`, a bounded runtime, a stable unit name derived from the job ID, and `journald` only as a secondary log; the OpenTeam journal remains canonical. Use equivalent `launchd` and Windows service/Task Scheduler adapters on macOS and Windows. If a robust service manager is unavailable, refuse one-click background update and fall back to the manual foreground command rather than pretending it is durable.

Control protocol

Use a versioned CLI protocol that works identically over local execution and SSH:

```
openteam update plan --target 1.3.0 --json
openteam update start --target 1.3.0 --request-id <uuid> --json
openteam update status --job <job-id> --after <sequence> --json
openteam update cancel --job <job-id> --json
openteam update recover --job <job-id> --json
```

`start` returns only after the durable worker accepted and fsynced the request. Repeating the same `request-id` with identical arguments returns the same job; reusing it with different arguments returns a conflict. `status` returns the current snapshot plus all retained events after a sequence. This works after UI reload, application restart, SSH reconnect, or server downtime. The public application API is used only to verify normal service health, not as the sole status path.

The host inspection and server version endpoint should expose:

- `installationId`
- release and updater protocol versions
- current version and channel
- supported target/updater range
- install directory fingerprint or project identity
- public origin(s)

Before showing a one-click action, the desktop requires an installation-ID match between the server it is displaying and the SSH/local host it will mutate.

Durable job record

Store a canonical append-only event file per job and a compact current snapshot. Minimum fields:

```
{
  "schemaVersion": 2,
  "jobId": "...",
  "requestId": "...",
  "sequence": 17,
  "status": "running",
  "phase": "health_verification",
  "fromVersion": "1.2.3",
  "targetVersion": "1.3.0",
  "releaseDigest": "sha256:...",
  "installationId": "...",
  "commitState": "not_committed",
  "rollbackCapability": "full_snapshot",
  "cancellable": false,
  "safeToCloseDesktop": true,
  "messageCode": "health.waiting",
  "displayMessage": "Checking that OpenTeam is ready",
  "startedAt": "...",
  "updatedAt": "...",
  "error": null
}
```

Every side effect uses two journal events: an intent event is flushed before the action, then a completion event is flushed after it. The atomic snapshot is derived from the journal. File rename alone is not the full durability contract; flush the file and parent directory at transaction boundaries. On worker start or host reboot, reconcile incomplete intents against observable state before doing anything else.

State machine and commit semantics

```
accepted
-> preflight
-> release_verification
-> artifact_staging
-> drain
-> recovery_point
-> configuration_switch
-> schema_migration
-> staged_start
-> health_verification
-> commit
-> resume_workers_and_writes
-> succeeded

Any failure before commit:
-> rollback_configuration
-> rollback_state_if_required
-> restart_previous
-> verify_previous
-> rolled_back | needs_attention
```

Cancellation is allowed only before drain begins.

After commit, automatic rollback is forbidden; report a post-commit incident.

Important details:

- Preflight validates installation identity, updater compatibility, upgrade path, release signature/metadata expiry, exact artifact hashes and sizes, Compose rendering, ports, Docker daemon, backup destination, Docker capacity, and current health.
- Artifact staging pulls every digest and inspects the resolved images before downtime. Image provenance verification can be added as policy defense in depth. Sigstore emphasizes that signatures prove an allowed identity signed a digest; consumers still need an explicit trust policy. See the [Sigstore threat model](#).
- Drain stops new runs, lets checkpointable work settle for a bounded interval, rejects new mutations with a clear maintenance response, and records unfinished work. Then stop worker, server, and computer in a defined order.
- The recovery point includes the database and any volume a pre-commit component or migration can mutate. Prefer filesystem or Docker-volume snapshots when supported; otherwise use private archives with integrity hashes. Treat these as rollback snapshots, not off-host disaster backups.
- Use a PostgreSQL custom-format archive (`pg_dump -Fc`) and validate it with `pg_restore --list`. PostgreSQL describes custom format as flexible, compressed by default, and inspectable by `pg_restore`. See `pg_dump`. Restore with `--exit-on-error` and preferably `--single-transaction` where practical; PostgreSQL documents that single-transaction restore is all-or-nothing. See `pg_restore`.
- Do not perform an automatic PostgreSQL major-version change inside the ordinary application update. PostgreSQL documents version-specific backup/restore constraints and recommends careful concurrent testing for major upgrades. See [PostgreSQL upgrade guidance](#).
- Use reviewed migration files and a migration ledger. Release metadata declares supported from versions and whether old binaries remain schema-compatible. Prefer expand/contract changes so code rollback usually does not require database restore. A destructive transition must remain behind maintenance until commit and requires a verified full recovery point.
- Staged start should start PostgreSQL, run migrations, start computer and server in maintenance/read-only mode, and leave the worker stopped. Run internal health and a small authenticated smoke test. Require multiple consecutive successes and exact target version/protocol values.
- Commit is a durable event written before enabling ordinary traffic. Then enable writes and start the worker. Once new work is accepted, rollback to the pre-update dump would discard valid writes, so failure handling changes from rollback to incident/recovery guidance.

Progress and user experience

The UI should be a timeline driven by machine-readable events, not raw subprocess output. Use stable phase/message codes and let the desktop own user-facing copy. Suggested high-level steps:

1. Checking this server
2. Verifying OpenTeam 1.3.0
3. Downloading components
4. Pausing active work
5. Creating a recovery point
6. Applying database changes
7. Starting OpenTeam
8. Running safety checks
9. Finishing the update

Show byte progress only when the denominator is known; otherwise show an indeterminate activity state. Never invent a percentage or ETA. Keep sanitized technical output under an expandable "Details" section with the job ID and local diagnostic path. Errors should distinguish:

- `rolled_back`: "Update failed. OpenTeam 1.2.3 was restored and is healthy."
- `needs_attention`: "Recovery did not complete. OpenTeam is stopped to protect your data."
- `post_commit_degraded`: "The update completed, but a service is degraded. No automatic rollback was attempted because new work may already exist."

Before start, show a native confirmation surface with exact host, installation name/ID, current and target versions, expected interruption, backup capability, and connection method. After the host accepts the job, say "You can close the desktop app; the update will continue." If the platform cannot provide a durable worker, say the opposite and do not call the flow one-click safe.

The term "computer" is overloaded. Rename the settings row from "Server & computer" to "Server runtime" or "Server stack," and describe the computer container separately from the Electron desktop app.

Desktop and server version sequencing

Treat desktop and server updates as one coordinated plan but two separate transactions:

1. Check and download the desktop update without installing it.
2. Ask the host updater to move the server stack to the version compatible with the downloaded desktop release.
3. Observe the durable server job to a committed success.
4. Restart the desktop into the already-downloaded version.

This avoids installing a client that cannot operate the old server while ensuring that the old desktop can still monitor the server through the stable updater protocol. The update UI and updater protocol must remain compatible across at least the supported client window.

Release metadata recommendation

Near term, add one signed release manifest that binds:

- release version, channel, publication time, and expiry
- minimum updater protocol/version
- allowed direct upgrade sources or minimum source version
- exact Compose digest and size
- exact image names, platform digests, SBOM/provenance references
- database migration ID/hash and rollback policy
- mutable-volume schema versions
- minimum disk estimate inputs
- known critical post-update checks

Verify the artifacts and GitHub build attestations in the consumer, not only produce them. GitHub notes that attestations establish provenance, not that an artifact is safe; verification and policy are required. See [GitHub artifact attestations](#).

Longer term, publish the same facts through a real TUF repository with a pinned initial root, threshold/offline root and targets roles, short-lived online timestamp metadata, and rollback/freeze tracking. Keep Sigstore provenance as a complementary build-identity signal. TUF answers "is this authorized and current for this channel?" while Sigstore/provenance answers "which workflow/build produced this digest?"

Alternatives considered

Keep the current foreground CLI and improve messages

This is the smallest change, but it cannot guarantee survival of desktop exit, SSH loss, or host reboot. Better copy does not repair transaction ownership. Use only as an interim/manual path.

Add an update endpoint to the existing OpenTeam server

This removes SSH setup but makes a remotely reachable application process capable of commanding Docker and replacing itself. It also disappears during the update. Without a separate management plane, it does not solve durability or observation. Not recommended.

Privileged updater container with Docker socket

This is easy to deploy but gives a network-adjacent container root-equivalent control over the host's Docker engine and must update itself. The security and lifecycle costs outweigh the convenience. Not recommended.

Separate HTTPS management daemon

This can eventually provide the smoothest experience, but it requires enrollment, mutual authentication, authorization, certificate rotation, rate limiting, audit, and a safe self-update story. It is a reasonable later transport over the same durable job engine, not a prerequisite. Start with local IPC plus SSH.

Prioritized implementation plan

P0: prerequisites for production-safe one-click updates

1. Introduce `update plan/start/status/recover` and a versioned job/event schema.
2. Launch updates under an OS service manager and make the job independent of Electron/SSH.
3. Implement write-ahead transitions, idempotent request IDs, startup reconciliation, and explicit `rolled_back` versus `needs_attention` outcomes.
4. Add maintenance/drain, staged start, and a durable commit point; do not run the worker or accept writes before commit.
5. Replace `prisma db push` with committed production migrations, baseline existing installs, and test every supported upgrade edge.
6. Inventory mutable state and implement a rollback snapshot contract for database plus affected volumes.
7. Enable GitHub immutable releases; publish drafts only after all assets exist; remove the replace-in-place release path.

P1: safety hardening and polished UX

1. Bind SSH/local host identity to the displayed OpenTeam installation ID.
2. Use custom-format PostgreSQL backups, integrity hashes, `pg_restore --list`, and periodic real restore tests.
3. Make capacity checks data-aware across the backup filesystem and Docker storage.
4. Add per-phase timeouts, watchdog heartbeats, cancellation only before drain, and sanitized diagnostic bundles.
5. Render a reconnectable timeline, native confirmation, clear commit/rollback results, and optional desktop notification on completion.
6. Add signed release compatibility fields and consumer-side provenance verification.

P2: advanced resilience and automation

1. Add migration rehearsal against a temporary restored database for risky releases or small installations.
2. Introduce TUF-based channel metadata and key-rotation/revocation procedures.
3. Offer opt-in scheduled patch updates with a maintenance window only after fault-injection evidence is strong.
4. Add a separately authenticated management transport if product demand justifies removing SSH.

Acceptance criteria and test strategy

A server update is safe only if every test ends in one of three explicit states: target version healthy; previous version restored and healthy; or `needs_attention` with services stopped to prevent further mutation and a deterministic recovery command.

Automate fault injection at every journal boundary:

- kill the desktop, SSH client, updater process, and Docker client independently
- disconnect the network during release fetch and image pull
- reboot the host during backup, configuration switch, migration, staged start, and rollback
- exhaust backup storage and Docker storage separately
- corrupt signature, digest, Compose document, migration, dump archive, and a mutable-volume archive

- point a valid SSH alias at the wrong OpenTeam instance
- make health return the wrong version, transient ready, permanent degraded, and queue/computer failures
- fail database restore and one volume restore independently
- retry the same request concurrently from two desktops
- update from every supported source version and architecture, not only N-1
- prove that no user or worker writes occur between recovery-point capture and commit

For realistic fixtures, seed data in PostgreSQL and every mutable volume. After success, verify version, migration ledger, component health, preserved data, and the ability to restart again. After rollback, verify old-version health and hashes/counts of the seeded state. Test backups by restoring them, not only by checking that a file is non-empty.

Suggested release gate metrics:

- 100 percent of injected pre-commit failures automatically reach healthy rollback or deterministic `needs_attention`
- 0 accepted mutations between drain completion and commit
- 0 false-success reports
- successful reconnect and event replay after desktop restart at every phase
- restore drill passes for the oldest supported upgrade source
- update duration and downtime recorded separately; no guessed UI ETA

Bottom line

Keep the existing CLI transaction and strict SSH transport, but move the transaction into a durable, host-owned, recoverable job. The desktop should plan, confirm, start, and observe through a stable protocol. Pair that with production migrations, a full-state rollback contract, a maintenance-protected commit point, and immutable/signed release metadata. This preserves the simplicity of a single-host Docker Compose product while making "Update from desktop" truthful even when the desktop disappears at the worst possible moment.



Material limitations

- This was a static code and documentation review plus focused unit tests; it did not perform a destructive live upgrade of a real installation.
- The worktree contains unrelated user changes, but the updater files reviewed here had no local diff.
- Exact launchd and Windows service mechanics should be prototyped on their target operating systems before finalizing the cross-platform supervisor interface.
- Full zero-downtime deployment is not the goal for a single-instance stateful stack. The recommendation optimizes for bounded, visible maintenance and data safety.

Claim-to-source ledger

- Electron IPC sender validation and renderer isolation: Electron, "Security," current documentation, <https://www.electronjs.org/docs/latest/tutorial/security>
- Detached OS-managed execution: systemd manual, "systemd-run(1)," current manual, <https://man7.org/linux/man-pages/man1/systemd-run.1.html>
- SSH non-interactive and host-key policy: OpenBSD, "ssh_config(5)," current manual, https://man.openbsd.org/ssh_config
- Compose readiness behavior: Docker, "Control startup and shutdown order in Compose" and "docker compose up," current documentation, <https://docs.docker.com/compose/how-tos/startup-order/> and <https://docs.docker.com/reference/cli/docker/compose/up/>
- Digest pinning and disconnect behavior: Docker, "docker image pull," current documentation, <https://docs.docker.com/reference/cli/docker/image/pull/>
- Docker persistent-volume backup/restore: Docker, "Volumes," current documentation, <https://docs.docker.com/engine/storage/volumes/>
- PostgreSQL dump, restore, and upgrade constraints: PostgreSQL Global Development Group, PostgreSQL 18/17 documentation, <https://www.postgresql.org/docs/current/app-pgdump.html>, <https://www.postgresql.org/docs/current/app-pgrestore.html>, and <https://www.postgresql.org/docs/17/upgrading.html>
- Production migration workflow: Prisma, "Prototyping your schema" and "Development and production," Prisma ORM v6 documentation, <https://www.prisma.io/docs/orm/v6/prisma-migrate/workflows/prototyping-your-schema> and <https://www.prisma.io/docs/orm/v6/prisma-migrate/workflows/development-and-production>
- TUF rollback/freeze/consistency model: The Update Framework, "Overview," "Roles and metadata," and "Security," <https://theupdateframework.io/docs/overview/>, <https://theupdateframework.io/docs/metadata/>, and <https://theupdateframework.io/docs/security/>
- Sigstore signature meaning and policy limits: Sigstore, "Verifying Signatures" and "Threat Model," <https://docs.sigstore.dev/cosign/verifying/verify/> and <https://docs.sigstore.dev/about/threat-model/>
- Immutable releases and attestation limitations: GitHub, "Immutable releases" and "Artifact attestations," <https://docs.github.com/en/code-security/concepts/supply-chain-security/immutable-releases> and <https://docs.github.com/en/actions/concepts/security/artifact-attestations>
- Repository implementation: OpenTeam at commit 4c29c3a2b8ba1c067055b3c46c8196c789eaff46, relevant files linked inline above, accessed September 4, 2026.